

Secure Network Application Development Using TCP

Computer Networks Laboratory

Student Details

Name: K. M. Swaroop

Roll Number: 323506402262

Course: Computer Networks Laboratory

Assignment: Assignment 7 – Secure Network Application Development Using TCP

Programming Language: Python 3

Operating System: Ubuntu 24.04 LTS

Virtualization Platform: Oracle VirtualBox

1. Objective

The objective of this assignment is to enhance the GUI-based multi-client TCP chat application developed in Assignment 6 by implementing practical security mechanisms. The application introduces user authentication, secure password storage using SHA-256 hashing, duplicate login prevention, input validation, failed login protection, session management, secure logging, and Wireshark verification. The focus is on improving application-level security while maintaining the existing networking functionality.

2. Introduction

Modern network applications require strong authentication and secure communication mechanisms to protect user information and prevent unauthorized access. Simply allowing users to connect without authentication exposes the application to various security risks.

In this assignment, the existing GUI-based chat application was enhanced with practical security features including password hashing, user authentication, login protection, session management, and secure logging. These additions improve the overall reliability and security of the application while preserving the original chat functionality.

3. Software Requirements

- Ubuntu 24.04 LTS
- Oracle VirtualBox
- Python 3

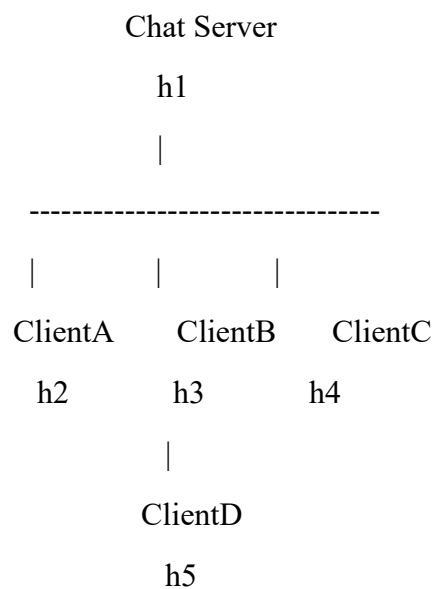
- Tkinter
- Socket Programming
- Threading
- hashlib
- CSV
- Mininet
- Wireshark

4. Hardware Requirements

- Intel/AMD Processor
- Minimum 8 GB RAM
- 20 GB Free Storage
- Internet Connection

5. System Architecture

The application follows a client-server architecture.



The server authenticates users before establishing a session. After successful authentication, users can participate in secure chat communication.

6. Security Features Implemented

6.1 User Authentication

A username and password-based authentication mechanism was implemented.

The server validates user credentials before allowing access to the chat system.

Only authenticated users are allowed to communicate.

6.2 Secure Password Storage

Passwords are never stored in plain text.

Instead, passwords are stored as SHA-256 hashes.

Example:

ClientA,password_hash

Hash generation:

```
hashlib.sha256(password.encode()).hexdigest()
```

This prevents attackers from obtaining actual passwords even if the user database is compromised.

6.3 Duplicate Login Prevention

The server maintains a list of currently logged-in users.

If the same username attempts to log in again while already connected, the login request is rejected.

Benefits:

- Prevents account sharing
- Prevents session hijacking
- Prevents conflicting sessions

6.4 Input Validation

The application validates:

- Empty username
- Empty password
- Invalid username
- Unsupported commands

- Oversized messages
- Invalid login attempts

This improves both security and application stability.

6.5 Failed Login Protection

After five consecutive failed login attempts, the user account is temporarily blocked for a fixed duration.

This protects the application against brute-force password attacks.

6.6 Session Management

Each authenticated user maintains an active session.

The application supports:

- Secure logout
- Session timeout
- Automatic removal from online users

Inactive sessions are terminated automatically after the configured timeout period.

6.7 Secure Logging

A separate security log records important security events.

Examples:

- Login Success
- Login Failed
- Duplicate Login
- Logout
- Session Timeout

Passwords are never written to the log file.

7. Components Reused from Assignment 6

The following components from Assignment 6 were reused without major modifications:

- TCP Socket Communication
- Multi-threaded Server

- GUI Chat Window
- Broadcast Messaging
- Private Messaging
- Online User List
- Chat History
- Logging System
- Background Message Thread

Only the authentication and security modules were added on top of the existing implementation.

8. Program Implementation

Server

The server performs the following operations:

1. Accept client connection
2. Receive username and password
3. Authenticate user
4. Prevent duplicate login
5. Validate user input
6. Start secure session
7. Receive messages
8. Broadcast messages
9. Handle private messaging
10. Maintain online users
11. Log security events
12. Handle logout
13. Handle session timeout

Client GUI

The GUI provides:

- Login Window
- Username Entry

- Password Entry
- Connect Button
- Chat Window
- Scrollable Chat Area
- Online Users Panel
- Message Entry
- Send Button
- Disconnect Button
- Connection Status Indicator

9. Testing

The application was tested using Mininet.

Topology:

```
sudo mn --topo single,5
```

Verification commands:

```
nodes
```

```
net
```

```
pingall
```

Testing scenarios:

- Successful login
- Invalid login
- Duplicate login
- Broadcast messaging
- Private messaging
- User join
- User leave
- Logout
- Session timeout

All tests executed successfully.

10. Wireshark Verification

Display Filter

`tcp.port == 5000`

The following events were captured:

- Client Login
- Failed Login
- Successful Authentication
- Broadcast Message
- Private Message
- Logout

The packet capture confirmed successful TCP communication and authenticated message exchange.

11. Experimental Results

The implemented application successfully:

- Authenticated valid users
- Rejected invalid credentials
- Prevented duplicate logins
- Protected passwords using SHA-256
- Validated user input
- Blocked repeated failed logins
- Managed active user sessions
- Logged security events
- Supported secure chat communication

12. Advantages

- Secure authentication
- Password hashing
- Multi-client communication
- Duplicate login prevention

- Input validation
- Session timeout
- Secure logging
- GUI-based interface
- User-friendly operation

13. Limitations

- Uses SHA-256 without salting
- Communication is not encrypted with TLS
- User database is CSV-based
- No role-based access control
- No two-factor authentication

14. Future Enhancements

Future improvements may include:

- SSL/TLS encryption
- AES encrypted messages
- Two-factor authentication
- Role-based authorization
- Database integration
- Password reset
- Email verification
- File sharing
- Group chat
- Voice and video communication

15. Conclusion

The Secure Network Application was successfully developed by extending the GUI-based chat application created in Assignment 6. The implementation introduced practical security mechanisms including user authentication, SHA-256 password hashing, duplicate login

prevention, failed login protection, input validation, session management, and secure logging while preserving the existing networking functionality.

The developed application demonstrated secure TCP-based communication between multiple authenticated users. Wireshark verification confirmed successful login, authenticated communication, and logout events. The assignment provided practical knowledge of authentication mechanisms, secure password storage, session management, and application-level security in network programming.